

Mini API HTTP avec Python

Aymane Es-salehy

7 avril 2026

1 Introduction

Ce travail pratique a pour objectif de concevoir une mini API HTTP en utilisant le module natif `http.server` en Python, ainsi que de développer un client capable d'interagir avec cette API.

L'objectif principal est de comprendre :

- le fonctionnement des requêtes HTTP (GET, POST),
- la gestion des erreurs,
- l'authentification par token,
- la communication client-serveur.

2 Description du serveur

Le serveur a été implémenté à l'aide de la classe `HTTPServer` et d'un gestionnaire personnalisé `BaseHTTPRequestHandler`.

2.1 Fonctionnalités principales

- **GET /health** : vérifie l'état du serveur et retourne un statut.
- **POST /documents** : permet de créer un document (authentification requise).

Les données sont stockées dans un dictionnaire Python simulant une base de données.

2.2 Gestion des erreurs

Le serveur gère plusieurs codes HTTP :

- 400 : requête invalide
- 401 : authentification échouée
- 404 : endpoint non trouvé
- 201 : création réussie

3 Lancement du serveur

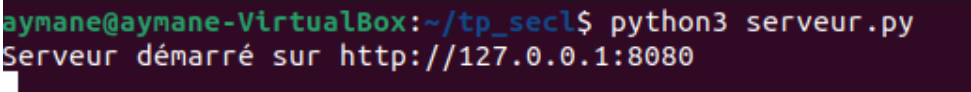
Pour démarrer le serveur :

```
python server.py
```

Le serveur écoute sur :

```
http://127.0.0.1:8080
```

3.1 Capture d'écran du serveur



```
aymane@aymane-VirtualBox:~/tp_secl$ python3 serveur.py
Serveur démarré sur http://127.0.0.1:8080
```

FIGURE 1 – Lancement du serveur

4 Description du client

Le client a été développé avec le module `urllib`.

4.1 Fonctionnalités

- Envoi de requêtes HTTP (GET, POST)
- Ajout des headers :
 - Content-Type : application/json
 - Authorization : Bearer token
 - X-Request-Id (identifiant unique)
- Gestion des erreurs HTTP et réseau

5 Lancement du client

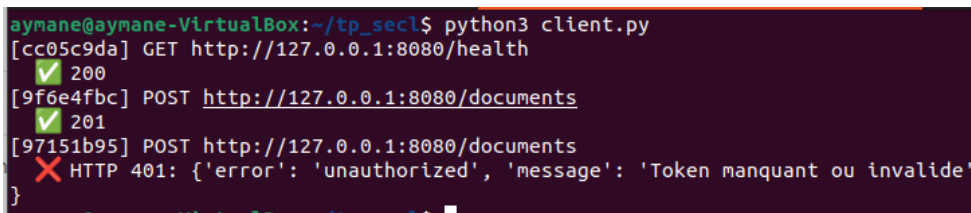
Pour exécuter le client :

```
python client.py
```

Le client teste :

- GET /health
- POST /documents
- POST sans authentification

5.1 Capture d'écran du client



```
aymane@aymane-VirtualBox:~/tp_secl$ python3 client.py
[cc05c9da] GET http://127.0.0.1:8080/health
✓ 200
[9f6e4fbc] POST http://127.0.0.1:8080/documents
✓ 201
[97151b95] POST http://127.0.0.1:8080/documents
✗ HTTP 401: {'error': 'unauthorized', 'message': 'Token manquant ou invalide'}
```

FIGURE 2 – Exécution du client

6 Notion de Retry

Une stratégie de retry avec backoff exponentiel a été proposée pour améliorer la robustesse.

Cependant, elle n'est pas utilisée systématiquement car :

- certaines requêtes (POST) ne sont pas idempotentes,
- cela peut entraîner des duplications.

7 Conclusion

Ce TP a permis de comprendre les bases de la création d'une API HTTP en Python sans framework.

Les notions importantes abordées sont :

- gestion des routes HTTP,
- validation des données,
- codes de statut HTTP,
- authentification,
- communication client-serveur.